

Results: what we measured and what it means

Thesis: *A multimodal framework for failure detection, adaptive recovery, and experience memory in autonomous web agents.* Prepared 2026-10-06 for the supervisor meeting.

Our system has two parts. **Task 1** asks: *can our trained model tell whether a web-agent step failed?* **Task 2** asks: *if we put that model on top of a real agent, does the agent finish more tasks?* Task 1 results are final. Task 2 results come from the completed paired run *web-v3* (4–6 October 2026; 42 real-website tasks, 2 repeats, 168 episodes). The base agent is **Browser Use 0.13.10 driven by Qwen2.5-VL-7B-Instruct**; "Ours" is the same agent with the proposed framework attached.

Table 1. Which backbone to use (trained on our data, full validation set)

Backbone	Step failure (MCC)	Step failure (Macro-F1)	Recovery worked? (MCC)	Recovery worked? (acc.)	Worth remembering? (MCC)
Qwen2-VL-2B	0.624	0.809	0.796	90.9 %	0.663
Qwen2.5-VL-7B (selected)	0.678	0.839	0.852	93.5 %	0.695
InternVL3.5-8B-HF	0.641	0.820	0.840	92.9 %	0.654

7,861 validation steps; 1,858 recovery cases; seed 42; one checkpoint per model, chosen by a rule fixed in advance.

What this means. All three models learn the job well from our data (MCC 0.62–0.68 on step failure; 0.80–0.85 on recovery). Qwen2.5-VL-7B is the best on every column, so it is the model we use in the agent. The gap between models is small, which is good news: our dataset works for different backbones, not just one.

How good is good? MCC runs from –1 to +1; 0 means guessing. Values around 0.65 mean the model gets most steps right in both directions (it catches real failures *and* does not cry wolf on good steps). Recovery checking is stronger (0.85): "did the fix work?" is an easier question than "did this step fail?".

Limit. These are single-seed validation numbers. The locked test split has not been used yet.

Table 2. Does training matter? Same model, same cases, trained vs. only prompted

Frozen subset of our validation data: 240 steps and 120 recovery cases. The "prompted" rows use the **same untrained model** and ask it to judge the step the way three public agents do (their self-check instructions, adapted). Unreadable answers count as wrong.

2a. "Did this step fail?" (240 cases)

How the step is judged	Model	Trained on our data?	MCC	Macro-F1	Correct on all cases	Readable answer
Browser Use-style prompt	Qwen2.5-VL-7B	no	0.000	0.237	11 %	87 / 240
Agent S2-style prompt	Qwen2.5-VL-7B	no	0.000	0.183	16 %	170 / 240
WebVoyager-style prompt	Qwen2.5-VL-7B	no	—	—	0 %	0 / 240
Browser Use-style prompt	Qwen2-VL-2B	no	0.000	0.206	25 %	228 / 240
Agent S2-style prompt	Qwen2-VL-2B	no	0.000	0.199	25 %	238 / 240
WebVoyager-style prompt	Qwen2-VL-2B	no	0.000	0.200	23 %	220 / 240
Our trained heads	InternVL3.5-8B	yes	0.558	0.768	80 %	240 / 240
Our trained heads	Qwen2.5-VL-7B	yes	0.649	0.820	85 %	240 / 240

2b. "Did the recovery work?" (120 cases)

How it is judged	Model	Trained?	MCC	Macro-F1	Correct on all cases	Readable answer
Browser Use-style prompt	Qwen2.5-VL-7B	no	−0.122	0.400	17 %	30 / 120
Agent S2-style prompt	Qwen2.5-VL-7B	no	−0.104	0.331	38 %	91 / 120
WebVoyager-style prompt	Qwen2.5-VL-7B	no	—	—	0 %	0 / 120
Browser Use-style prompt	Qwen2-VL-2B	no	0.000	0.329	46 %	112 / 120
Agent S2-style prompt	Qwen2-VL-2B	no	0.000	0.333	50 %	120 / 120
WebVoyager-style prompt	Qwen2-VL-2B	no	0.000	0.335	46 %	109 / 120
Our trained heads	InternVL3.5-8B	yes	0.884	0.942	94 %	120 / 120
Our trained heads	Qwen2.5-VL-7B	yes	0.841	0.916	92 %	120 / 120

What this means. This is the most important table for Task 1. Every prompted row sits at **MCC 0** (pure guessing), and the prompted models often cannot even give a readable yes/no (WebVoyager-style: zero readable answers out of 240). The **same** Qwen2.5-VL-7B, after training on our data, jumps to **MCC 0.65** with a readable answer on every case. Same model, same cases, only training differs. So the improvement comes from **our dataset and training**, not from the size or brand of the model.

Why this is good news, not bad. If the untrained model had already scored well, our dataset would add nothing. The fact that it scores zero is what makes the training (and the dataset) necessary. It also explains why plain agents loop: their own self-check ("did my last step work?") is no better than a coin flip.

One limit to state. The prompted rows copy those agents' *self-check instructions*; they are not the full agents running live. So the claim is "prompted self-assessment does not work", not "those agents are bad".

Small note. Table 2 numbers (0.649 / 0.841) differ a little from Table 1 (0.678 / 0.852) because Table 2 uses 240 / 120 selected cases and Table 1 uses the full validation set. Both are reported as what they are.

Table 3. Our detector next to published detectors (context only; different data)

Model	Size	Trained for this?	Data it was tested on	Accuracy
GPT-4o, best prompt [1]	undisclosed	no	failure traces of [1]	83.8 %
InternVL2-76B, best prompt [1]	76 B	no	same	82.4 %
Gemini-1.5-pro, best prompt [1]	undisclosed	no	same	79.7 %
GPT-4o, default prompt [1]	undisclosed	no	same	74.4 %
Gemini-1.5-flash, default prompt [1]	undisclosed	no	same	61.5 %
Ours: Qwen2.5-VL-7B + trained heads	7 B, 4-bit	yes	our validation set	≈83 % (MCC 0.678)

What this means. A 7B open model trained on our data lands in the same range as prompted GPT-4o and a 76B model, at about a tenth of the size and running locally. **But** these are different test sets, so this table is background, not a race. The honest sentence is "comparable magnitude on different data". Never write "beats GPT-4o".

Table 4. Agent level: does adding failure handling help a real agent?

System	What is added to the base agent	Actor model	Benchmark	Base agent	With system	Gain
Multimodal Auto Validation [2]	vision validator + retry	GPT-4-class	WebVoyager	76.2	81.2	+5.0
BacktrackAgent [3]	error detection + backtracking	Qwen2-VL (fine-tuned)	Auto-UI (mobile)	21.6	29.7	+8.1
WebCoach [4]	memory coach	Qwen-VL-7B	WebVoyager	32.8	28.8–31.1	–1.7 to –4.0
WebCoach [4]	memory coach	Qwen-VL-32B	WebVoyager	49.5	54.7–57.1	+5 to +8
Devil's Advocate [5]	reflection before acting	GPT-4	WebArena	19.8	23.5	+3.7
OSCAR [6]	re-planning on state change	GPT-4	GAIA	22.4	28.7	+6.3
SkillWeaver [7]	learned skills	GPT-4o	WebArena	22.6	29.8	+7.2
Ours, development tasks (20)	trained failure detection + recovery + memory	Browser Use + Qwen2.5-VL-7B base	real websites, 40 pairs	70.0	87.5	+17.5
Ours, held-out tasks (22)	same	same	real websites, 44 pairs	75.0	84.1	+9.1
Ours, all tasks (42)	same	same	real websites, 84 pairs	72.6	85.7	+13.1

What this means. Published systems that add checking or recovery to an agent usually gain **+4 to +8 points**. Ours gains +13.1 points overall and +9.1 on tasks never seen during design, so it sits at the upper end of that range, with a 7B open actor rather than GPT-4. These are different benchmarks, so the comparison is of relative gains, not absolute rates.

The WebCoach 7B row is a warning that supports us. With a 7B actor, extra advice made the agent *worse*; it only helped bigger actors. We saw exactly this in our first run (web-v2: ours 20/38 vs. baseline 27/38) and redesigned the recovery layer (v3) to remove it. The final run (Tables 6–8) shows that it did.

Two things in our evaluation that none of the cited papers do: **paired comparison with a significance test**, and a **held-out task set** that was never looked at while designing the system.

Table 5. Why the base agent fails (our traces vs. published analyses)

Source	Main failure causes
AutoWebGLM [9]	hallucination 44 %, poor visual recognition 28 %, misread task 20 %, pop-ups 8 %
WebSuite [10]	simple actions (click, type) succeed 76–85 %; "find/filter" tasks only 40–44 %
Ours (web-v2 baseline, 11 failures)	repeating a useless action 9 (82 %): scrolling past the page end 5, clicking/typing the same box 4; false "done" 1; broken output 1

What this means. Our base agent's main problem is that it keeps doing something that has no effect while believing it worked (the "hallucination" class in [9]). That is exactly what a before/after failure detector can see, which is why our approach targets it. The two rare cases (declaring "done" without acting; malformed output) produce no step to judge and are outside what any step-level detector can fix.

Task 2 results: Browser Use with and without the proposed framework

Setup. 42 real-website tasks from domains in our dataset: 20 *development* tasks (inspected while the recovery layer was designed) and 22 *held-out* tasks (written and verified before any run, never inspected). Each task was run twice by each system, back-to-back, giving 84 pairs. The base agent is **Browser Use 0.13.10 driven by the frozen Qwen2.5-VL-7B-Instruct**. "Browser Use + Ours" is the same agent, prompts, 15-step budget and start pages, with our framework attached (P1 failure detection, adaptive recovery, experience memory). A task counts as completed when the environment sees the target page URL; the agent never sees this rule.

Table 6. Task completion of Browser Use with and without the proposed framework

Task set	Pairs	Browser Use	Browser Use + Ours	Δ (pts)	p
Development	40	70.0	87.5	+17.5	0.039
Held-out	44	75.0	84.1	+9.1	0.219
All	84	72.6	85.7	+13.1	0.007

Completion rate in %. Actor: Qwen2.5-VL-7B-Instruct (frozen) in both columns. p: exact two-sided sign test on discordant pairs.

What this means. This table answers the main question of Task 2: does the framework make Browser Use finish more tasks? Over all 84 paired episodes, Browser Use alone completed 72.6 % of tasks; with our framework it completed 85.7 %, a gain of 13.1 points. Out of every 100 tasks Browser Use would fail on about 27; with our framework it fails on about 14, so **roughly half of its failures are removed**. The result is statistically significant ($p = 0.007$). On the development tasks the gain is +17.5. On the held-out tasks, never inspected during design, the gain is +9.1 and not significant on its own, because only six pairs differ. The direction is the same on tasks the system had never seen, so the improvement is not an artefact of tuning to known tasks.

Table 7. Paired outcomes

Task set	Helped	Hurt	Same
Development	8	1	31
Held-out	5	1	38
All	13	2	69

Helped: Browser Use failed, Browser Use + Ours completed. Hurt: the reverse.

What this means. Each task is run by both systems back-to-back, so pairs can be read directly. In 69 of 84 pairs the result was the same, as expected: the framework does nothing unless the failure detector fires. Among the 15 pairs that differed, ours won 13 and lost 2. **When the framework changes Browser Use's outcome, it improves it in 87 % of cases.** The two losses (Wikipedia "black hole", jQuery API) are examined in the error analysis.

Table 8. Rescue and harm rates

Task set	Rescue rate	Harm rate
Development	67 (8/12)	4 (1/28)
Held-out	45 (5/11)	3 (1/33)
All	57 (13/23)	3 (2/61)

Rescue: share of Browser Use failures completed with Ours. Harm: share of Browser Use successes failed with Ours. Values in %.

What this means. A recovery system must fix failures and leave successes alone. The framework rescued 57 % of the episodes Browser Use failed (67 % development, 45 % held-out) while disturbing only 3 % of the episodes Browser Use would have completed. The low harm rate matters as much as the rescue rate: our first design (web-v2) had a 22 % harm rate and 0 % rescue, because it slowed the actor and sent misleading labels. The redesigned layer removed almost all harm while adding the rescues.

Table 9. Cost per episode

Measure	Browser Use	Browser Use + Ours
Mean time (s)	220	175
Mean steps, both completed (n = 59)	3.08	3.32

What this means. The framework adds work per step (an assessment after each action; extra actor calls after a detected failure), yet the average episode is **20 % shorter**. When Browser Use fails it usually spends all 15 steps repeating a useless action; ours breaks the loop or finishes. On the 59 pairs both completed, ours used 0.24 steps more on average, the price of occasional recovery detours. The framework costs almost nothing when Browser Use is doing well and saves time when it is not.

Table 10. Recovery activity of the framework on top of Browser Use (84 episodes)

Measure	Count
Episodes with at least one recovery	31
Recovery attempts	55
Option selections by the actor	69
Repeat-guard blocks	30
Uncertain alarms suppressed ($P < 0.9$)	22
Memory records written	34
Memory records shown	10

What this means. The failure detector opened a recovery in 31 of 84 episodes; in the other 53 it stayed silent and Browser Use's input was identical to the baseline's. When it fired, the actor picked an offered option 69 times and was stopped from repeating a failed action 30 times, which is where the loop-breaking comes from. The confidence gate held back 22 uncertain alarms, protecting the harm rate. The memory stored 34 experiences and showed 10 on repeat visits; its effect is small here because each site is visited only twice.

Table 11. Per-task outcome (two repeats; Y completed, N failed)

Task	Split	Browser Use	Browser Use + Ours
Alan Turing	Dev	N Y	Y Y
Eiffel Tower	Dev	N Y	Y Y
Renew passport	Dev	N Y	Y Y
Bank holidays	Dev	N Y	Y Y
EPA climate change	Dev	N N	Y Y
IETF RFCs	Dev	N N	Y N
TED talks	Dev	N N	N Y
jQuery API	Dev	Y Y	Y N
Debian news	Dev	N N	N N
11 other tasks	Dev	Y Y	Y Y
cPanel pricing	Held-out	N N	Y Y
Driving licence	Held-out	N N	N Y
Community portal	Held-out	N Y	Y Y
Marie Curie	Held-out	N Y	Y Y
Black hole	Held-out	Y N	N N
EPA recycle	Held-out	N N	N N
NOAA weather	Held-out	N N	N N
15 other tasks	Held-out	Y Y	Y Y

What this means. Three patterns appear. **Consistent rescues:** on EPA climate change and cPanel pricing Browser Use failed both times and ours succeeded both times, so these are not chance. **Variance rescues:** on Alan Turing, Eiffel Tower, passport, bank holidays, community portal and Marie Curie, Browser Use failed once and succeeded once while ours succeeded both times; the framework removes Browser Use's bad days. **Beyond the framework:** Debian news, EPA recycle and NOAA weather failed in all four runs; on these pages the target link shares no words with the goal, so the recovery layer cannot point the actor to it. The two losses show the remaining risk: in both, the actor left the correct path and then produced invalid output, leaving no executed step for the detector to judge.

Summary for the supervisor

Our dataset trains any of three vision-language backbones to judge web-agent steps well (MCC 0.62–0.68), with Qwen2.5-VL-7B the best. On identical cases, the same model without training scores zero, so the gain is from our data, not the model. The trained 7B detector is in the same accuracy range as prompted GPT-4o reported elsewhere, at a tenth of the size. Attached to Browser Use with a 7B actor, the framework raised task completion on 42 real-website tasks from 72.6 % to 85.7 % (84 paired episodes; 13 helped, 2 hurt; $p = 0.007$), rescued 57 % of the base agent's failures while harming 3 % of its successes, and shortened episodes by 20 %. The gain also holds on 22 held-out tasks never seen during design (+9.1 points). Published systems that add checking or recovery gain +4 to +8 points, and small actors are known to be harmed by naive advice, which our redesigned recovery layer avoided. Remaining failures concentrate on pages whose target link shares no words with the goal and on actor output errors that produce no step to judge; these are the subject of the error analysis that follows.

References: [1] [arXiv:2509.14382](#) · [2] [arXiv:2410.00689](#) · [3] [arXiv:2505.20660](#) · [4] [arXiv:2511.12997](#) · [5] [arXiv:2405.16334](#) · [6] [arXiv:2410.18963](#) · [7] [arXiv:2504.07079](#) · [9] [arXiv:2404.03648](#) · [10] [arXiv:2406.01623](#). Published numbers were transcribed from the papers' tables; please check them against the original PDFs before use in the thesis.